Compte rendu de TP

ESP32 : LED et feux de carrefour
avec écran OLED

Étudiants	**Mohamed Aziz Khouni** Eya Jounaidi Eya Htiwech	**Établissement**	Faculté de Gabès, LISI2
Carte	TTGO ESP32 + OLED SSD1306	**Logiciel**	Arduino IDE

Sommaire

1. Objectifs
2. Matériel et logiciels
3. Notions théoriques
4. Expérience 1 : Allumer et éteindre une LED
5. Expérience 2 : Feux de carrefour
6. Difficultés rencontrées
7. Conclusion

1. Objectifs

- Prendre en main la carte **TTGO ESP32** et l'**Arduino IDE**.
- Piloter une **LED** avec une sortie numérique (`digitalWrite`).
- Afficher des informations sur un **écran OLED** grâce au bus **I2C**.
- Réaliser un **feu de carrefour** à trois LEDs (rouge, jaune, vert) avec affichage de la couleur active.

2. Matériel et logiciels

Élément	Détail
Carte	TTGO ESP32 avec écran OLED intégré (128x64, SSD1306)
Alimentation et programmation	Câble micro-USB relié à l'ordinateur
Montage	Breadboard et fils de connexion (jumpers)
LEDs	1 rouge (expérience 1), puis rouge, jaune et verte (expérience 2)
Résistances	Une résistance de protection en série avec chaque LED
Logiciel	Arduino IDE, bibliothèques Adafruit_GFX et Adafruit_SSD1306

3. Notions théoriques

3.1 Pourquoi une résistance avec la LED ?

Une LED a une résistance interne très faible : sans résistance en série, le courant devient trop fort et la LED (ou la broche de l'ESP32) peut être détruite. La valeur se calcule avec la loi d'Ohm :

R = (V_alimentation - V_LED) / I

Exemple avec une LED rouge : V_alimentation = 3,3 V, V_LED = 2 V environ, courant voulu = 10 mA. On obtient R = (3,3 - 2) / 0,010 = 130 ohms. On choisit une valeur normalisée proche, par exemple 220 ohms, ce qui donne un courant d'environ 6 mA, suffisant pour allumer la LED sans risque.

3.2 Le bus I2C

L'écran OLED communique avec l'ESP32 avec seulement deux fils :

Signal	Rôle	Broche utilisée
SDA	Données	GPIO 21
SCL	Horloge	GPIO 22

L'écran répond à l'adresse **0x3C**.

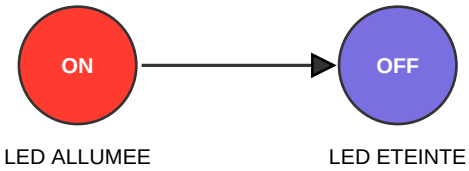
3.3 Sortie numérique

`pinMode(broche, OUTPUT)` configure une broche en sortie. Ensuite, `digitalWrite(broche, HIGH)` met la broche à 3,3 V (LED allumée) et `digitalWrite(broche, LOW)` la met à 0 V (LED éteinte).

4. Expérience 1 : Allumer et éteindre une LED

4.1 Principe

La LED est branchée sur la **GPIO 2** (`LED_BUILTIN` dans le code) à travers une résistance, l'autre patte étant reliée à **GND**. Toutes les secondes, le programme change l'état de la LED et met à jour l'écran.



Cycle : LED allumée 1 s, puis éteinte 1 s, et ainsi de suite.

4.2 Câblage

Fil	De	Vers
Violet	GPIO 2 de l'ESP32	LED (via la résistance)
Noir	GND de l'ESP32	Autre patte de la LED

4.3 Montage réalisé

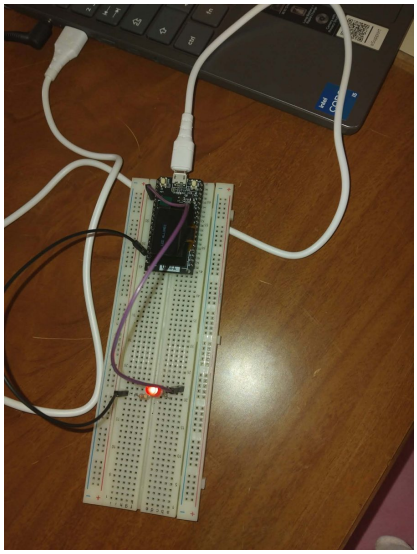


Figure 1 : montage de l'expérience 1

4.4 Code

```
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define LED_BUILTIN 2

Adafruit_SSD1306 oled(128, 64, &Wire, -1);

void setup() {
  Serial.begin(115200);
  pinMode(LED_BUILTIN, OUTPUT);
  Wire.begin(21, 22);
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  oled.setTextColor(SSD1306_WHITE);
  WiFi.mode(WIFI_STA);
  delay(500);
}

void loop() {
  // --- LED ALLUMEE ---
  digitalWrite(LED_BUILTIN, HIGH);
  oled.clearDisplay();
  oled.setCursor(0, 0);
  oled.setTextSize(1);
  oled.println("--- Etat LED ---");
  oled.setTextSize(2);
  oled.println("LED ALLUMEE");
  oled.display();
  delay(1000);

  // --- LED ETEINTE ---
  digitalWrite(LED_BUILTIN, LOW);
  oled.clearDisplay();
  oled.setCursor(0, 0);
  oled.setTextSize(1);
  oled.println("--- Etat LED ---");
  oled.setTextSize(2);
  oled.println("LED ETEINTE");
  oled.display();
  delay(1000);
}
```

4.5 Explication du code

Instruction	Rôle
#include <...>	Charge les bibliothèques pour l'I2C et l'écran OLED
Adafruit_SSD1306 oled(128, 64, &Wire, -1)	Crée l'objet écran de 128x64 pixels
pinMode(LED_BUILTIN, OUTPUT)	Configure la broche 2 en sortie
Wire.begin(21, 22)	Démarre l'I2C (SDA = 21, SCL = 22)
oled.begin(SSD1306_SWITCHCAPVCC, 0x3C)	Initialise l'écran à l'adresse 0x3C
oled.clearDisplay()	Efface l'écran avant d'écrire
oled.setCursor(x, y), oled.setTextSize(n)	Règle la position et la taille du texte
oled.println("...")	Écrit le texte dans la mémoire de l'écran
oled.display()	Envoie la mémoire à l'écran (sans cet appel, rien ne s'affiche)
delay(1000)	Pause de 1 seconde

4.6 Résultats

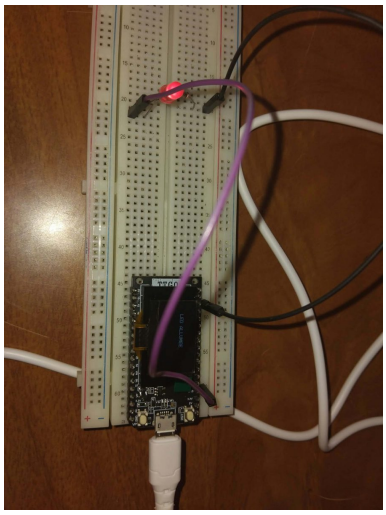


Figure 2 : LED allumée, l'écran affiche LED ALLUMEE

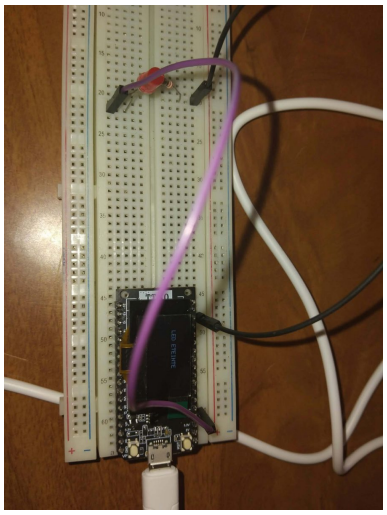


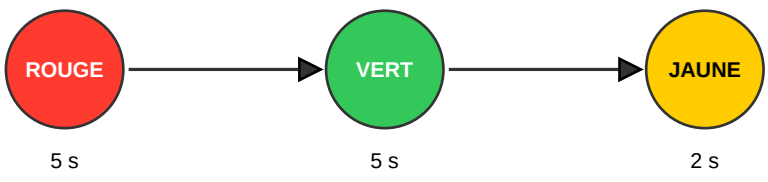
Figure 3 : LED éteinte, l'écran affiche LED ETEINTE

La LED clignote toutes les secondes et l'écran suit toujours l'état réel de la LED.

5. Expérience 2 : Feux de carrefour

5.1 Principe

On utilise trois LEDs, chacune avec sa résistance et sa propre broche. À chaque instant, **une seule LED est allumée**. L'écran affiche la couleur active en grand et un compte à rebours du temps restant.



Cycle : rouge, vert, jaune, puis retour au rouge.

Phase	LED allumée	Durée	Affichage OLED
Rouge	Rouge	5 s	ROUGE + temps restant
Vert	Verte	5 s	VERT + temps restant
Jaune	Jaune	2 s	JAUNE + temps restant

5.2 Câblage

LED	Broche ESP32	Résistance
Rouge	GPIO 25	oui (en série)
Jaune	GPIO 26	oui (en série)
Verte	GPIO 27	oui (en série)
Masse commune	GND	-

5.3 Montage réalisé

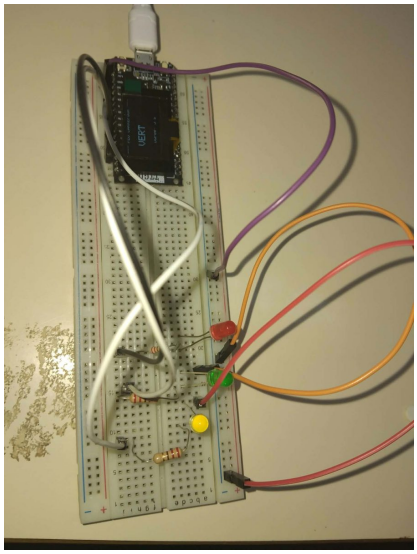


Figure 4 : montage du feu de carrefour (LED verte allumée)

5.4 Code

```
/*
  TP ESP32 - Feux de carrefour avec écran OLED
  Carte : TTGO ESP32 (OLED SSD1306 128x64, I2C : SDA = 21, SCL = 22)

  Câblage (chaque LED en série avec une résistance de ~220 ohms) :
    LED rouge -> GPIO 25
    LED jaune -> GPIO 26
    LED verte -> GPIO 27
    L'autre patte de chaque résistance/LED -> GND
*/

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ----- Broches des LEDs -----
#define LED_ROUGE 25
#define LED_JAUNE 26
#define LED_VERT  27

// ----- Durées des phases (en secondes) -----
#define DUREE_ROUGE 5
#define DUREE_VERT  5
#define DUREE_JAUNE 2

Adafruit_SSD1306 oled(128, 64, &Wire, -1);

// Allume une seule LED à la fois
void allumer(bool rouge, bool jaune, bool vert) {
  digitalWrite(LED_ROUGE, rouge);
  digitalWrite(LED_JAUNE, jaune);
  digitalWrite(LED_VERT,  vert);
}

// Affiche la couleur active et le temps restant sur l'écran OLED
void afficher(const char* couleur, int restant) {
  oled.clearDisplay();

  oled.setTextSize(1);
  oled.setCursor(0, 0);
  oled.println("--- FEU CARREFOUR ---");

  oled.setTextSize(3);              // Couleur en grand
  oled.setCursor(0, 22);
  oled.println(couleur);

  oled.setTextSize(1);
  oled.setCursor(0, 54);
  oled.print("Reste : ");
  oled.print(restant);
  oled.println(" s");

  oled.display();
}

// Exécute une phase : allume la LED, puis compte à rebours à l'écran
void phase(bool rouge, bool jaune, bool vert, const char* couleur, int duree) {
  allumer(rouge, jaune, vert);
  for (int t = duree; t > 0; t--) {
    afficher(couleur, t);
    delay(1000);
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(LED_ROUGE, OUTPUT);
  pinMode(LED_JAUNE, OUTPUT);
  pinMode(LED_VERT,  OUTPUT);
  allumer(false, false, false);

  Wire.begin(21, 22);
  if (!oled.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("Erreur : ecran OLED introuvable");
    while (true);                   // On bloque si l'écran n'est pas détecté
  }
  oled.setTextColor(SSD1306_WHITE);
```

```
}

void loop() {
  phase(true,  false, false, "ROUGE", DUREE_ROUGE);
  phase(false, false, true,  "VERT",  DUREE_VERT);
  phase(false, true,  false, "JAUNE", DUREE_JAUNE);
}
```

5.5 Explication du code

Élément	Rôle
#define LED_ROUGE 25 ...	Donne un nom à chaque broche, ce qui rend le code lisible
#define DUREE_ROUGE 5 ...	Règle la durée de chaque phase en un seul endroit
allumer(rouge, jaune, vert)	Règle les trois LEDs d'un coup, pour qu'une seule soit allumée
afficher(couleur, restant)	Écrit la couleur (grande taille) et le temps restant sur l'écran
phase(...)	Allume la LED d'une couleur, puis fait le compte à rebours seconde par seconde
if (!oled.begin(...))	Vérifie que l'écran est détecté, sinon affiche une erreur dans le moniteur série
loop()	Enchaîne les phases : rouge, vert, jaune, puis recommence

Les fonctions `allumer`, `afficher` et `phase` évitent de répéter le même code pour chaque couleur.

5.6 Résultat

> Sur la figure 4, la LED **verte** est allumée et l'écran affiche **VERT**. Le cycle se répète indéfiniment : rouge, vert, jaune, rouge...

6. Difficultés rencontrées

Problème	Solution
La LED restait allumée après le passage à « LED ETEINTE »	Ajout de digitalWrite(LED_BUILTIN, LOW) dans la partie « éteinte »
L'écran restait noir	Vérification de l'adresse I2C (0x3C), des broches SDA/SCL et de l'appel à oled.display()
Risque de brûler les LEDs	Utilisation d'une résistance en série avec chaque LED

7. Conclusion

Ce TP nous a permis de découvrir la programmation de l'ESP32 avec l'Arduino IDE. Nous avons appris à :

- piloter des LEDs avec une sortie numérique et les protéger par une résistance ;
- afficher du texte sur un écran OLED grâce au bus I2C ;
- enchaîner plusieurs états dans `loop()` pour simuler un feu de carrefour ;
- organiser le code avec des fonctions pour le rendre plus clair.

Les deux montages ont fonctionné comme prévu. Ce travail peut être amélioré avec un feu piéton, un bouton poussoir ou un clignotement du jaune.